

An 84-Format Numeric Catalog with Bit-Exact Conformance Vectors: A Vendor-Neutral Reference for FP8, BF16, MXFP4, and Microscaling Formats

Dmitrii Vasilev*
Trinity S³AI

2026-06-08 (preprint)

Abstract

Numeric format proliferation in machine learning hardware – FP8 (E4M3 and E5M2), BF16, MXFP4, microscaling block formats, and dozens of research variants – has outpaced the availability of vendor-neutral, bit-exact reference material. Engineers porting models across accelerators encounter silent divergences that are difficult to diagnose without a shared ruler.

This paper describes a catalog of 84 numeric formats spanning 13 families, and a suite of six bit-exact conformance packs covering GF16, MXFP4 element, BF16, FP8 E4M3, FP8 E5M2, and E8M0 block scale. Each pack is a self-contained JSON document with a SHA-256 fingerprint, a shared row schema, and an anchor vector that encodes $3.0 - \frac{1}{\varphi^2} = 3$ [1] – as a cross-pack sanity check. Packs are cross-validated against `ml_dtypes` 0.5.4 (Google/JAX); any divergence is documented explicitly and interpreted as a spec-permitted interpretation gap rather than hidden. The work is framed as registry filling: it does not propose new formats, make model-accuracy claims, or assert superiority over any vendor’s implementation. All artifacts are publicly available at <https://github.com/gHashTag/t27> under an open license.

1 Introduction

Imagine a machinist who needs to fit a component to a lathe specification, but the measuring ruler in hand uses units that differ subtly from those in the drawing. The part may look correct; the divergence only surfaces under load. The same scenario plays out in ML accelerator firmware: two chips may both claim “FP8 E4M3 support”, yet differ silently in how they handle the overflow case for an input such as 1000.0 – one saturates to max-finite 448.0, the other flips to NaN. The OCP Microscaling specification [16] permits both choices. Without a shared bit-exact reference, a ported model may produce numerically different results that are difficult to isolate.

This paper describes two artifacts designed to serve as that shared ruler.

Contribution 1: An 84-format numeric catalog. The `t27` catalog enumerates 84 numeric formats across 13 families (Section 3). Each entry carries a uniform schema: bit layout, bias, infinity/NaN policy, saturation policy, max-finite value, min-normal, min-subnormal, and a claim-status tag (Verified / Empirical_fit / Open_conjecture / Risk / Retracted). The catalog is stored as

*github.com/gHashTag; ORCID [0009-0008-4294-6159](https://orcid.org/0009-0008-4294-6159).

a single source of truth and cross-compiled to Markdown, JSON, Python, Rust, C, and TypeScript via a template tool.

Contribution 2: Six bit-exact conformance packs. The packs (Section 5) cover the six formats most commonly seen in current production hardware and research pipelines: GoldenFloat 16 (GF16), MXFP4 element, BF16, FP8 E4M3, FP8 E5M2, and E8M0 block scale. Two packs (GF16 and MXFP4) are already live in the `tt-lang-t27` PyPI package v0.3.1; the remaining four are introduced in the current pre-release, available at <https://github.com/gHashTag/tt-lang-t27/pull/6>.

What this paper is not. This paper presents no model-accuracy benchmarks, no novel format proposals, and no performance comparisons between vendors. Readers seeking FLOP throughput analysis or quantization accuracy results should consult the separate literature.

Roadmap. Section 2 surveys the relevant standards landscape. Section 3 describes the catalog design. Section 4 defines the conformance pack methodology. Section 5 presents each of the six packs in turn. Section 6 provides an IEEE P3109 cross-walk. Section 7 discusses the interpretation gap as a design feature. Section 8 covers reproducibility and provenance. Section 9 outlines future work.

2 Background and Prior Work

The proliferation of low-precision floating-point formats in machine learning hardware has been characterized, even in a popular 2017 essay, as “the wild west of computer arithmetic” [9]. Nearly a decade later the landscape is richer (more vendors, more block formats, more research variants), yet vendor-neutral, bit-exact reference material has not kept pace.

2.1 Floating-Point Standards

IEEE 754-2019 [15] defines binary interchange formats (binary16, binary32, binary64, binary128) and the rounding, overflow, and NaN rules that govern them. BF16 (brain float 16) is not in the 2019 revision; it arose informally at Google Brain and is now supported across Intel, AMD, ARM, and NVIDIA hardware, sharing the exponent range of FP32 (8 exponent bits, bias = 127) with a 7-bit mantissa.

FP8 formats followed a similar informal-then-formal trajectory. Noune et al. [14] surveyed 8-bit numerical formats for deep neural networks in 2022, motivating the E4M3 and E5M2 variants subsequently adopted into OCP MX and IEEE P3109.

The OCP Microscaling (MX) specification v1.0 [16] introduces block formats in which groups of 32 elements share a common E8M0 scale factor. The element types include MXFP4 (S1E2M1), MXFP6 (E2M3 and E3M2), MXFP8 (E4M3 and E5M2), and MXINT8. OCP MX explicitly permits two overflow policies for FP8 E4M3: saturation to max-finite (used by the tt-metal and AMD implementations) and overflow to NaN (used by JAX/TPU).

NVIDIA’s NVFP4 [7] is a recent 4-bit variant that pairs an MXFP4-style S1E2M1 element with a 16-element block (smaller than OCP MX’s 32-element block) and uses an FP8 E4M3 block scale rather than E8M0. At the time of writing, NVFP4 is documented in NVIDIA technical-blog form and Blackwell/Rubin software stacks; it is not yet covered by an open inter-vendor specification.

M^2 XFP [8] generalizes microscaling further by allowing mixed element precisions within a block, with hardware feasibility studied in the ASPLOS ’26 timeframe.

IEEE P3109 [18] is an active working group standardizing 8-bit and 4-bit floating-point formats for AI workloads. Its v3.2.0 Interim Report defines Binary8p3se and Binary4p1sf (among others) and a `StandardOperations.yaml` catalogue of approximately 80 operations across seven categories.

2.2 Existing Reference Implementations

ml_dtypes [17] (Google/JAX) is a Python/C++ library offering reference implementations of `bfloat16`, `float8_e4m3fn`, `float8_e5m2`, `float8_e8m0fnu`, and several other formats. It is the ground-truth oracle used throughout this work.

P3109 FLoPS [4] is a Lean 4 formalization of the P3109 semantics, providing proof-checked coverage of key operations.

Py chop [10] (Carson & Chen, 2025) emulates a wide family of low-precision arithmetics in Python, including FP8 variants, posits, and customizable (E, M, bias) tuples. It targets ML and scientific-computing workloads and is complementary to the present catalog: Py chop emulates operation-layer behavior; the packs in this paper pin down representation-layer bit patterns.

libtakum [12] is a reference C library for the takum arithmetic family of Hunhold [2, 13], providing a usable baseline for cross-implementing the catalog’s Posit/Unum III cluster against an independent oracle.

torch.float8 (PyTorch) and **jax.dtypes** expose FP8 types at the framework level but do not publish bit-vector test suites independent of hardware execution.

MX evaluation studies [5] measure accuracy impact of microscaling quantization in transformer workloads.

2.3 The Gap

No single vendor-neutral artifact currently covers FP8 E4M3, FP8 E5M2, BF16, MXFP4 element, NVFP4 element [7], GoldenFloat 16, and E8M0 block scale in one schema with: (a) bit-exact encode/decode vectors, (b) SHA-256-anchored provenance, (c) explicit documentation of each divergence from the reference implementation, and (d) a human-readable cross-walk to IEEE P3109. This work fills that registry gap for the six representation-layer packs that are most immediately deployable; NVFP4 is discussed as a near-term Track 2 candidate (Section 9) and as a second interpretation gap (Section 7).

3 Catalog Design

3.1 84 Formats Across 13 Clusters

The `t27` catalog contains 84 formats organized into 13 named clusters. Table 1 shows the cluster names and format counts. The sum of counts is exactly 84; this is a continuously enforced catalog invariant (CI-01, Section 3.4).

3.2 One-Row-Per-Format Schema

Each catalog entry carries the following fields:

- **name** – canonical identifier (ASCII, no spaces)

Table 1: The 84 formats across 13 clusters (T1).

Cluster	Representative formats	Count
IEEE754 binary	binary16, binary32, binary64, binary128, binary80	5
IEEE754 decimal	decimal32, decimal64, decimal128	3
MLLowPrecision	BF16, TF32, FP8 E4M3, FP8 E5M2, FP8 E3M4, FP6, FP4, NF4	8
GoldenFloat	GF4 through GF256 (phi-anchored variants)	16
Posit / Unum III	Posit8, Posit16, Posit32, takum8, takum16, ...	8
OCP MX	MXFP4, MXFP6_E2M3, MXFP6_E3M2, MXFP8_E4M3, E8M0_block	5
LNS	LNS8, LNS16 variants	4
IntegerFixed	INT2, INT4, INT8, INT16, UINT4, UINT8, UINT16, FXP16	8
HistoricalVendor	IBM hex float, DEC VAX G, Cray single, NVIDIA TF32, ...	10
Theoretical	E0M7, E7M0, E1M6, E6M1 boundary cases	4
Compression/scaling	NF4 block, E8M0 block, SF8, RFP8	4
Extended	binary256, bfloat32, bfloat128	3
QuantTuned	Q-BF16, adaptive-FP8	2
Total		84

- **bits** – total bit width
- **exp** – exponent field width in bits
- **mant** – mantissa field width in bits (0 for E8M0-style)
- **bias** – exponent bias
- **has_inf** – boolean
- **has_nan** – boolean
- **saturation_policy** – SatFinite, OvfInf, or OvfNaN
- **max_finite** – largest representable finite value (f64)
- **min_normal** – smallest positive normal value (f64)
- **min_subnormal** – smallest positive subnormal (f64; **null** if none)
- **cluster** – one of the 13 cluster labels
- **claim_status** – Verified / Empirical_fit / Open_conjecture / Risk / Retracted

3.3 Claim-Status Taxonomy

Verified: format spec is backed by a published standard (IEEE, OCP) or by a proof-checked reference (P3109 FLoPS Lean). **Empirical_fit:** derived by fitting the observed bit layout of a hardware product without an independently published spec. **Open_conjecture:** proposed generalization awaiting external validation. **Risk:** spec reference exists but the catalog encoding may contain errors not yet caught by the test suite. **Retracted:** previously included; removed after a conflicting authoritative source was identified.

Table 2: 15 catalog invariants (CI-enforced) (T2).

ID	Invariant	Check
CI-01	Total format count equals 84	<code>sum(cluster_counts) == 84</code>
CI-02	No name collisions	<code>len(names) == len(set(names))</code>
CI-03	Bit-width consistency	<code>1 + exp + mant == bits</code> for standard layout
CI-04	Bias range	<code>bias <= 2**(exp-1) - 1</code>
CI-05	Saturation policy present	field not null for every entry
CI-06	Max-finite positive	<code>max_finite > 0</code>
CI-07	Min-normal $\leq$ max-finite	ordering preserved
CI-08	Cluster label in enum	no unlisted cluster names
CI-09	Claim status in enum	no unlisted claim-status values
CI-10	No format with 0 bits	<code>bits >= 2</code>
CI-11	SHA-256 anchor present	each pack header carries fingerprint field
CI-12	Anchor vector present	each pack contains <code>anchor_*</code> vector
CI-13	Anchor decodes to 3.0	<code>decode(anchor_bits) == 3.0</code>
CI-14	Codegen targets compile	CI matrix runs Python + Rust import tests
CI-15	No duplicate SHA-256	pack fingerprints are globally unique

3.4 Catalog Invariants

Fifteen invariants are checked on every commit. Selected invariants are listed in Table 2.

3.5 Codegen Path

A single Jinja2 template tool reads the canonical JSON catalog and emits per-language output files: Markdown (human-readable table), JSON (API export), Python dataclasses, Rust structs with `serde` derives, C header (`#define` constants), and TypeScript enum literals. All generated files are committed to the repository at <https://github.com/gHashTag/t27> and rebuilt on every push via a GitHub Actions matrix.

4 Conformance Pack Methodology

4.1 Shared Row Schema

Every conformance pack is a JSON array of vectors, each row conforming to the schema shown in Table 3.

4.2 Pack Header

In addition to the vector array, each pack file carries a header object with the following fields:

- Format spec quadruple: $(E, M, \text{bias}, \text{infNaN policy})$
- Saturation policy
- Max-finite value
- SHA-256 self-fingerprint (computed over the canonical JSON serialization)
- `ml_dtypes` version anchor
- Anchor identity reference: $\phi^2 + 1/\phi^2 = 3$ (arXiv:2606.05017)

Table 3: Shared row schema for all conformance packs (T3).

Field	Type	Description
<code>name</code>	string	human-readable test-case identifier
<code>input_f64</code>	number	input value as a double-precision float
<code>input_f64_hex</code>	string	IEEE 754 hex encoding of the input (f32 or f64)
<code><fmt>_bits_hex</code>	string	target-format bit pattern, hex
<code><fmt>_bits_int</code>	integer	same bit pattern as an unsigned integer
<code>decoded_f64</code>	number	result of <code>decode(encode(input))</code>
<code>decoded_f64_hex</code>	string	IEEE 754 hex encoding of decoded value
<code>abs_error</code>	number	$ \text{input} - \text{decoded} $; always shown (never hidden)
<code>category</code>	string	zero / normal / subnormal / inf / nan / overflow / underflow / rounding / anchor / transcendental

4.3 Anchor Vector

Every pack contains at least one vector named `anchor_*` that encodes the value 3.0. The motivation is the identity

$$\varphi^2 + \frac{1}{\varphi^2} = 3, \quad (1)$$

where $\varphi = (1 + \sqrt{5})/2$ is the golden ratio. This identity is presented and contextualized in the Gold-enFloat preprint [1] as a numerically grounded L_2 anchor. The value 3.0 is exactly representable in all six pack formats (it falls in the normal range with zero mantissa error for all six layouts), making it a reliable single-line sanity check across packs.

Formally: for any pack format F , if $\text{decode}_F(\text{encode}_F(3.0)) \neq 3.0$, a fundamental implementation error is present.

4.4 Verification Steps

Each pack is checked by two independent procedures:

1. **Round-trip self-check.** For each vector: `decode(encode(input)) = decoded`, with the stored `abs_error` consistent with the deviation.
2. **Cross-check against `ml_dtypes` 0.5.4.** Where a corresponding `ml_dtypes` type exists, the pack’s bit patterns are compared against the `ml_dtypes` encoding of the same inputs. Every divergence is recorded in the pack header’s `divergences` list and described in this paper.

Honest treatment of absolute error is a non-negotiable design principle. Every vector where the decoded value differs from the input carries a nonzero `abs_error`; no value is suppressed or rounded to zero to make match statistics look better.

5 The Six Conformance Packs

Table 4 gives a summary of all six packs.

Full SHA-256 fingerprints (verbatim from the manifest):

- **GF16:** see repository (SHA-256 not yet pinned in v0.4.0-pre manifest)

Table 4: Six packs at a glance (T4).

Pack	Layout	Vecs	ml_dtypes match	Status	SHA-256 (16)
GF16	S1E5M10 phi-anchored	21	n/a (no ml_dtypes equiv.)	LIVE v0.3.1	see repo
MXFP4	S1E2M1 block element	12	n/a	LIVE v0.3.1	86c99d6f72375d75
BF16	S1E8M7 bias=127, RTE	21	21/21	NEW v0.4.0-pre	320c1850b4846745
FP8 E4M3	S1E4M3 bias=7, SatMax	16	15/16 (Sec. 7)	NEW v0.4.0-pre	fff0c30f8e6bee22
FP8 E5M2	S1E5M2 bias=15, Ovflnf	17	17/17	NEW v0.4.0-pre	66cd7be1500ec800
E8M0 block	E8M0 scale-only, no sign	11	OCP MX v1.0 aligned	NEW v0.4.0-pre	b211f1a863f71fd7

- **MXFP4**: 86c99d6f72375d751df4c74897904a0a36cff52e8d60cbfef5d58b71625d4b2f
- **BF16**: 320c1850b484674546785791b1c22d76feb4ea748c6669ffb633e5455d822b8a
- **FP8 E4M3**: fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf97dba0539973f0a5e89d
- **FP8 E5M2**: 66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f22a75d02f7842f23a56
- **E8M0 block**: b211f1a863f71fd7c5e02e512efff0255ebcc51521311186e01cb9992e4464bd

5.1 GF16 – GoldenFloat 16-bit

GF16 is a 16-bit format using layout S1E5M10 with a phi-rotation of the representable range. It is described and motivated in the GoldenFloat preprint [1]. The pack contains 21 vectors covering zero, normal values, the anchor 3.0 (encoding the identity $\varphi^2 + 1/\varphi^2 = 3$, Eq. (1)), subnormals, and overflow behavior. Because ml_dtypes does not implement a GF16 type, this pack has no cross-validation partner; its vectors are verified by the round-trip self-check only. GF16 has been live in the `tt-lang-t27` PyPI package since v0.3.1.

5.2 MXFP4 Element – OCP Microscaling 4-bit

MXFP4 element uses layout S1E2M1 (1 sign, 2 exponent, 1 mantissa bit) with saturation-to-finite overflow policy, as specified in OCP MX v1.0 [16]. Within a block, 32 such elements share an E8M0 scale factor. The element pack covers 12 vectors: the 15 representable finite values plus zero and the saturation case. ml_dtypes does not expose an MXFP4 element type at the time of writing; the pack is verified by round-trip self-check and compared against the OCP MX v1.0 value table. SHA-256: 86c99d6f72375d751df4c74897904a0a36cff52e8d60cbfef5d58b71625d4b2f.

5.3 BF16 – Brain Float 16

BF16 uses layout S1E8M7 with bias = 127, round-to-nearest-even (RTE), and IEEE 754-style handling of infinity and NaN. It occupies the upper 16 bits of an FP32 word, so conversion to/from FP32 is a simple truncation (with rounding). The pack contains 21 vectors, including:

- Positive and negative zero
- Positive and negative infinity (preserved exactly)
- Quiet NaN (preserved with payload)
- Smallest positive normal and subnormal
- Largest finite BF16 ($\approx 3.39 \times 10^{38}$)
- Two RTE midpoint cases (round-to-even behavior)
- Overflow of FP32 max into BF16 $+\infty$ (`abs_error =  $+\infty$`)
- Underflow of FP32 min-subnormal to BF16 $+0$
- Non-exact constants φ and $1/\varphi$ with nonzero `abs_error`
- The anchor vector at 3.0 (exact, `abs_error = 0`)

All 21 vectors match `ml_dtypes.bfloat16` (Google/JAX 0.5.4): 21/21. SHA-256: 320c1850b484674546785791b1c22d76feb4ea748c6669ffb633e5455d822b8a.

BF16 exhibits high inter-vendor agreement; Google `bfloat16`, Intel `BFLOAT16`, ARM `BFloat16`, and NVIDIA TF32-paired BF16 share the same IEEE 754-style sub/inf/NaN semantics with round-to-nearest-even on the lower 16 bits of FP32. No notable divergences were observed in the 21 boundary cases tested.

5.4 FP8 E4M3 – Eight-bit Float with Four-bit Exponent

FP8 E4M3 uses layout S1E4M3 with bias = 7. In the OCP MX variant (used here), infinity is replaced by additional finite values, and NaN is encoded as bit pattern `0x7F` (or `0xFF` for negative). The format thus has no $+\infty$, giving a max-finite value of 448.0.

The pack contains 16 vectors. 15 of 16 match `ml_dtypes.float8_e4m3fn` exactly. The single documented divergence is the overflow case for input 1000.0, detailed in Table 5 and discussed in Section 7. SHA-256: fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf97dba0539973f0a5e89d.

Table 5: FP8 E4M3 overflow interpretation gap for input 1000.0 (T5).

Input	Implementation	Bits	Decoded	Policy
1000.0	this pack (tt-metal/AMD convention)	0x7E	448.0 (max-finite)	saturate-to-max
1000.0	<code>ml_dtypes</code> 0.5.4 (JAX/TPU convention)	0x7F	NaN	overflow-to-NaN

Both choices are permitted by OCP MX v1.0. See Section 7.

5.5 FP8 E5M2 – Eight-bit Float with Five-bit Exponent

FP8 E5M2 uses layout S1E5M2 with bias = 15 and retains full IEEE 754-style infinity and NaN. Max-finite is 57344.0. The pack contains 17 vectors covering the complete boundary suite (zero, normals, subnormals, $\pm\infty$, NaN, overflow, underflow, RTE midpoints, and the anchor 3.0). All 17 vectors match `ml_dtypes.float8_e5m2` exactly: 17/17. SHA-256: 66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f22a75d02f7842f23a56.

5.6 E8M0 Block Scale – OCP Microscaling Scale Format

E8M0 is a scale-only format used as the shared block exponent in OCP MX blocks. It carries no sign bit and no mantissa – only 8 exponent bits representing powers of 2 in the range $[2^{-127}, 2^{127}]$. The special pattern `0xFF` encodes NaN (used to indicate an uninitialized or invalid scale). The pack contains 11 vectors covering representative scale values, the NaN sentinel, and the anchor 3.0 (which encodes to the closest representable power-of-two scale, $2^1 = 2$, with a documented nonzero `abs_error`). Vectors were regenerated against `ml_dtypes.float8_e8m0fnu` (Google/JAX 0.5.4) following OCP MX v1.0 semantics. SHA-256: `b211f1a863f71fd7c5e02e512efff0255ebcc51521311186e01cb9992e4464bd`.

6 IEEE P3109 Cross-Walk

IEEE P3109 [18] is an active working group standardizing floating-point arithmetic for AI applications. Its v3.2.0 Interim Report defines a family of configured formats parameterized by $(E, M, \text{saturation})$. The broader case for explicit, bit-level conformance testing in industrial floating-point practice is made by Wintersteiger [11] at ARITH 2025; the packs in this paper are an instance of that pattern targeted specifically at the AI numeric-format registry. Where machine-checked semantics exist, e.g. the P3109 FLoPS Lean 4 development [4], the cross-walk in Table 6 is the bridge between proof-checked spec and bit-exact test data.

Table 6 maps the six packs to P3109 v3.2.0 configured formats.

Table 6: P3109 v3.2.0 cross-walk for the six packs (T6).

Our format	P3109 name	Match	Key difference	Pack
FP8 E4M3	Binary8p3se	Close	Saturation: ours SatMax vs. P3109 OvfInf; finite-only NaN encoding	<code>fp8_e4m3</code>
MXFP4 element	Binary4p1sf	Direct	Block structure orthogonal; element matches	<code>mxfp4</code>
GF16	(none)	No match	P3109 does not address 16-bit phi-anchored formats	<code>gf16</code>
BF16	(none)	No match	P3109 focuses on 4/8-bit; BF16 is 16-bit	<code>bf16</code>
FP8 E5M2	(none)	No match	Binary8p2se would correspond but is absent in v3.2.0	<code>fp8_e5m2</code>
E8M0 block	(none)	Orthogonal	P3109 does not define a scale-only format	<code>e8m0_block</code>

6.1 Direct Matches

Binary8p3se $\leftrightarrow$ FP8 E4M3. P3109 Binary8p3se specifies S1E4M3 with OvfInf saturation. The OCP MX v1.0 FP8 E4M3 variant used in this pack employs SatMax instead. The difference is exactly the overflow interpretation gap documented in Table 5. Aside from this saturation policy choice, the bit layouts and bias are identical.

Binary4p1sf $\leftrightarrow$ MXFP4 element. P3109 Binary4p1sf specifies S1E2M1 with SatFinite – identical to the MXFP4 element layout in OCP MX v1.0. The only structural difference is that OCP MX wraps elements in 32-element blocks sharing an E8M0 scale factor, a block dimension that P3109 does not address in v3.2.0.

6.2 Partial and Non-Matches

FP8 E5M2 would map to a hypothetical Binary8p2se, which is absent from P3109 v3.2.0 Profiles. GF16 and BF16 are outside the 4/8-bit scope that P3109 currently addresses. E8M0 is a scale-only format orthogonal to P3109’s representation layer.

6.3 Operational Coverage

P3109’s `StandardOperations.yaml` enumerates approximately 80 operations across seven categories: Classification (8), Comparison (7), Extrema (10+), Projection rounding (6 modes), Math arithmetic (10), Math transcendental (≈ 25), and Block operations (40+).

The current suite (v0.1) covers only the *representation layer* – encode/decode bit-exactness. Track 2 (target Q3 2026) will extend coverage to the operation layer, at minimum NearestTiesTo-Even rounding for Add, Multiply, and FMA across all six formats, with proof-checked semantics taken from P3109 FLoPS [4] as the formal anchor.

7 Discussion: The Interpretation Gap as Ruler Value

A conformance suite earns its keep not when all vectors match, but when it exposes a divergence that would otherwise be invisible. Two such cases are worth naming explicitly: the FP8 E4M3 overflow gap (Section 7.1) and the block-structure gap between MXFP4 and NVFP4 (Section 7.2).

7.1 Gap A: FP8 E4M3 Overflow Policy

The FP8 E4M3 overflow case (input = 1000.0) is the canonical example.

The OCP MX v1.0 specification [16] states that for inputs exceeding max-finite (448.0 for E4M3), implementations may either saturate to max-finite or produce NaN. Two mature, production-quality implementations make different choices:

- **tt-metal (Tenstorrent) / AMD convention:** saturate to max-finite. Bit pattern 0x7E, decoded value 448.0. This pack adopts this convention.
- **JAX/TPU convention (ml_dtypes 0.5.4):** overflow to NaN. Bit pattern 0x7F, decoded value NaN.

Neither choice is a bug. Both are compliant with OCP MX v1.0. The divergence is a documented spec-permitted interpretation gap.

The practical implication is significant for compiler and test-harness authors. Any cross-vendor port of an FP8 E4M3 computation must either: (a) select one policy explicitly and document it, or (b) carry both vectors in its golden-reference test suite, accepting that overflow-range inputs will produce differing results on different hardware.

This is precisely what a conformance pack is designed to expose. A test suite that compares only “do the outputs match on this hardware?” would never see this divergence – both implementations pass their own tests. A shared bit-exact reference makes the gap visible.

7.2 Gap B: 4-bit Block Structure (MXFP4 vs. NVFP4)

A second class of interpretation gap arises one level up, at the block structure rather than the element bit pattern. OCP MX MXFP4 and NVIDIA NVFP4 [7] share the same S1E2M1 element

Table 7: MXFP4 vs. NVFP4 block-structure parameters.

Parameter	OCP MX MXFP4	NVIDIA NVFP4
Element layout	S1E2M1 (4 bits)	S1E2M1 (4 bits)
Mantissa bits (element)	1	1
Block size	32 elements	16 elements
Scale format	E8M0 (8-bit)	FP8 E4M3 (8-bit)
Scale exponent bits	8 (pure exponent)	4
Scale mantissa bits	0	3
Scale dynamic range	2^{-127} to 2^{127}	$\approx 2^{-9}$ to 448
Scale granularity per decade	1 (power-of-two only)	8 (3-bit mantissa)
Bits/element including scale	$4 + 8/32 = 4.25$	$4 + 8/16 = 4.50$

layout (so the element packs are bit-identical at the 4-bit level), yet wrap that element in differently-shaped blocks with differently-quantized scale fields. Table 7 summarizes the parameter divergence.

Three structural consequences follow from the parameter table:

1. **Different scale resolution.** E8M0 (MXFP4) quantizes the block scale to a power of two; NVFP4’s FP8 E4M3 scale offers $2^3 = 8$ mantissa codes per binade and so resolves intra-block dynamic range eight times more finely within its representable range.
2. **Different scale range.** E8M0 spans approximately 2^{254} binades (subject to reserved codes); FP8 E4M3 saturates at 448 and underflows below $\approx 2^{-9}$. A tensor whose per-block scale naturally lies outside the FP8 range is representable in MXFP4 but not in NVFP4 without re-scaling at a higher level.
3. **Different effective bit budget.** Per-element storage is 4.25 bits for MXFP4 (32-element block) versus 4.50 bits for NVFP4 (16-element block). The two formats are not directly bit-comparable; any compression-ratio comparison must account for this 5.9% overhead delta in NVFP4.

A tensor stored as MXFP4 and the same tensor stored as NVFP4 may therefore agree on every element bit-pattern yet differ in decoded value, because the per-block scale they multiply by uses a different (and differently quantized) scale format. This is a structural interpretation gap rather than a value gap: element bit-exactness does not imply tensor-decoded equality.

The ruler reading is symmetric to Gap A. Both MXFP4 and NVFP4 are conforming implementations of a sensibly-designed 4-bit block format; they simply make different block-structure choices. Cross-vendor deployment of 4-bit weights requires explicit declaration of which block format is in use (block size + scale format), not just the element layout. Element-only declarations (“the model uses MXFP4-shaped 4-bit weights”) are insufficient.

The present catalog covers the MXFP4 side of this pair as a Tier-1 pack and lists NVFP4 as a near-term Track 2 candidate (Section 9.1). Documenting the structural gap is the first step; a sister NVFP4 pack with a matching schema and a deliberate cross-pack divergence vector at a representative scale-quantization boundary will close it.

7.3 General Pattern: Spec-Permitted Choice as a Ruler Reading

Honesty norm: every vector in every pack where the decoded value differs from the input carries a nonzero `abs_error`. Overflow to $\pm\infty$ shows `abs_error = Inf`; underflow to zero shows the actual

magnitude of the underflowed value. No `abs_error` field is suppressed or rounded to zero to improve match statistics.

8 Reproducibility and Provenance

8.1 Source Repositories

- **gHashTag/t27** (<https://github.com/gHashTag/t27>): the single source of truth (SSOT) for the catalog and Tier-1 conformance packs. All JSON catalog files, pack files, invariant checks, and codegen templates live here.
- **gHashTag/tt-lang-t27** (<https://github.com/gHashTag/tt-lang-t27>): PyPI mirror. Version 0.3.1 is live on PyPI (GF16 and MXFP4 packs included). Version 0.4.0-pre, adding the four new packs described in this paper, is available in [PR #6](#).
- **gHashTag/tt-trinity-corona** (<https://github.com/gHashTag/tt-trinity-corona>): Tier-2 silicon oracle context for post-silicon audit on GF180MCU; one-line mention here for completeness.

8.2 Ground-Truth Tool

The primary oracle for all cross-validation is `ml_dtypes` 0.5.4 [17] (Google/JAX), available at https://github.com/jax-ml/ml_dtypes. The specific types used are: `ml_dtypes.bfloat16`, `ml_dtypes.float8_e4m3fn`, `ml_dtypes.float8_e5m2`, and `ml_dtypes.float8_e8m0fnu`.

8.3 Anchor Fingerprint

The anchor identity $\varphi^2 + 1/\varphi^2 = 3$ (Eq. (1)), as formalized in the GoldenFloat preprint [1], has the following canonical SHA-256 fingerprint:

218403e344779c890f302ad2c70af21fb765060dd794d793c7eacc1ef8f80e6b

This fingerprint covers the canonical UTF-8 encoding of the identity string and serves as an out-of-band check that the correct anchor paper is being cited.

8.4 Pack Provenance Table

Table 8 lists the repository path, branch/PR, and full SHA-256 for each pack.

8.5 Manifest

The file `MANIFEST_v0.4.0-pre.json` in the `gHashTag/t27` repository records all six pack SHA-256 values, the `ml_dtypes` version anchor, and the P3109 alignment reference in a single machine-readable document. Downstream consumers can verify pack integrity by recomputing the SHA-256 of the canonical JSON file and comparing against the manifest entry.

Table 8: Pack-to-provenance mapping (T7).

Pack	Repo	Branch / PR	Full SHA-256
GF16	gHashTag/t27	main (v0.3.1)	see repo (not pinned in v0.4.0-pre manifest)
MXFP4	gHashTag/t27	main (v0.3.1)	86c99d6f72375d751df4c74897904a0a36cff52e8d60c bfe5d58b71625d4b2f
BF16	gHashTag/t27	PR #6 (v0.4.0-pre)	320c1850b484674546785791b1c22d76feb4ea748c666 9ffb633e5455d822b8a
FP8 E4M3	gHashTag/t27	PR #6 (v0.4.0-pre)	fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf9 7dba0539973f0a5e89d
FP8 E5M2	gHashTag/t27	PR #6 (v0.4.0-pre)	66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f 22a75d02f7842f23a56
E8M0 block	gHashTag/t27	PR #6 (v0.4.0-pre)	b211f1a863f71fd7c5e02e512efff0255ebcc51521311 186e01cb9992e4464bd

9 Future Work

9.1 Track 2: Full 84-Pack Suite

The six packs in this paper cover the formats most immediately relevant to production ML hardware. Track 2 (target Q3 2026) will extend coverage to all 84 catalog formats for which reference implementations are available, including:

- NVIDIA NVFP4 [7] (S1E2M1 element, 16-element block, FP8 E4M3 block scale) – closing the structural gap documented in Section 7.2.
- Remaining MLLowPrecision entries (FP6 variants, FP4, NF4).
- Posit/Unum III types via the libtakum [12] oracle.
- GoldenFloat variants GF4 through GF256.

9.2 Operation-Layer Conformance

The current suite covers the representation layer only. Track 2 will add operation-layer vectors aligned with the P3109 `StandardOperations.yaml` subset, beginning with NearestTiesToEven rounding for Add, Multiply, and FMA across all six current formats. This will allow compiler and hardware teams to validate not only that they encode/decode correctly, but that their arithmetic operations agree with the standard at the bit level.

9.3 Round-Trip Fuzzing

A property-based fuzzer will complement the hand-crafted boundary vectors. The fuzzer will generate random FP32 inputs, apply encode/decode for each format, and assert the round-trip property and `abs_error` consistency. This is particularly valuable for formats with complex saturation behavior.

9.4 Open Invitations

The most useful immediate next step is for an independent maintainer to take any single conformance pack, run it against their own implementation, and report divergences as GitHub issues on `gHashTag/t27`. Specifically:

- **ml_dtypes** [17]: confirm the 21/21 BF16 match and the 15/16 + 17/17 FP8 results against the latest 0.5.x release.
- **OCP MX** working group: validate MXFP4 element and E8M0 block scale vectors against the MX v1.0 reference table.
- **IEEE P3109** editors: cross-check the Binary8p3se / Binary4p1sf rows in Table 6 against the v3.2.x interim.
- **NVIDIA NVFP4** [7]: confirm the 16-element block / FP8 E4M3 scale parameters used in Section 7.2.
- **Pychop** [10], **libtakum** [12]: oracles for the Track 2 operation-layer and Posit/Unum III sub-suites respectively.
- **IREE, vLLM, llama.cpp, onnxruntime**: integrators most likely to be affected by the FP8 E4M3 overflow gap in cross-vendor deployments.

Any new divergence found is a feature of the ruler, not a failure of the suite.

The conformance packs, catalog schema, and codegen templates are open-licensed with the intent that they become a shared community resource for vendor-neutral numeric format registry work.

10 Limitations

The present catalog and conformance suite are deliberately scoped, and we state the boundary conditions explicitly so that downstream users can decide which claims to rely on.

Element-layer, not operation-layer. All six Tier-1 packs verify *encode/decode round-trip* behavior at the element level: a value is encoded to a bit-pattern, decoded back, and the result is compared against the ground-truth oracle. The packs do not verify operation-layer semantics — multiplication, accumulation, activation, or fused matmul-accumulate kernels. Operation-layer conformance (Track 2, Section 9.2) is acknowledged as the natural next layer but is out of scope for this preprint.

Single ground-truth oracle. The Tier-1 packs use **ml_dtypes** [17] as the reference implementation, with one well-documented divergence on FP8 E4M3 overflow (Section 7.1). An independently-implemented oracle — for example, **libtakum** [12] for Posit/Unum III or **Pychop** [10] for the Track 2 op layer — is not yet wired into the verification harness. Discrepancies traceable to a single-oracle bias cannot currently be ruled out for formats outside the BF16 / FP8 / MXFP4 / E8M0 subset.

Catalog coverage is asymmetric. The 84-row catalog (Section 3) covers 13 format clusters, but the depth of coverage is not uniform. IEEE-754 binary, BF16, FP8 (E4M3/E5M2), MXFP4, and the GoldenFloat GFN family are covered with full row schemas, claim-status taxonomy, and matching Tier-1 conformance packs where applicable. Posit/Unum III, logarithmic number systems (LNS), NF4, BitNet, TF32, and FP6 rows are present but currently lack Tier-1 packs; their claim-status fields are populated but not yet verified end-to-end against an oracle. This is by design — the catalog is a registry first, a verification harness second — but it means absence of a conformance pack should not be read as a quality claim.

NVFP4 documented but not packed. Gap B (Section 7.2) is documented at the structural level with a parameter table, but a matching Tier-1 NVFP4 pack is not yet included in this preprint. Quantitative cross-pack divergence vectors at representative scale-quantization boundaries are listed as the first Track 2 deliverable (Section 9.1).

No claim of optimality. The catalog does not claim that any included format is best for any downstream task. Comparisons are about *interpretation under a spec*, not about accuracy, throughput, energy, or model quality. Benchmarks that rank formats by accuracy or perplexity are outside the scope of this work; the present contribution is a vendor-neutral reference rather than a competitive evaluation.

Honest abs_error, not zero abs_error. The honesty norm of Section 7.3 requires that every vector with a nonzero decode error reports the actual magnitude. This makes the suite *look worse* than a suite that masks overflow-to-Inf or underflow-to-zero as “match.” Consumers comparing the present results against suites that suppress such fields should normalize the comparison before drawing conclusions.

Reproducibility envelope. All SHA-256 anchors, repository commits, and pack manifests reported in Section 8 were verified at the time of writing. Long-term reproducibility depends on the upstream availability of `ml_dtypes`, the OCP MX specification, and the IEEE P3109 interim document; if any of these become unavailable, the verification harness would need a mirror layer not currently implemented.

None of these limitations alter the central claim of the paper: that an 84-format catalog with six conformance packs, an IEEE P3109 cross-walk, and two documented interpretation gaps is now a usable, reproducible vendor-neutral reference. They circumscribe what the artifact *is* from what it is not.

Acknowledgments

The author thanks the maintainers of `ml_dtypes` (Google/JAX) for providing the ground-truth reference implementation, and the OCP Microscaling working group for publishing the MX v1.0 specification in open-access form. This work was carried out at Trinity S³AI. The author thanks the Tenstorrent tt-metal reviewers @amahmudTT and @rtawfik01 for substantive feedback on related FP8 conformance threads that informed the framing of Section 7. No external financial support was involved in the preparation of this preprint.

References

- [1] D. Vasilev, “GoldenFloat: A phi-anchored numeric format family and the identity $\varphi^2 + 1/\varphi^2 = 3$,” arXiv:2606.05017, 2026. <https://arxiv.org/abs/2606.05017>
- [2] C. Hunhold, “Takum arithmetic: A new paradigm for low-precision numerics,” arXiv:2412.20273, 2024. <https://arxiv.org/abs/2412.20273>
- [3] C. Park, J.-H. Lim, S. Nagarakatte, “ProofWright: Towards verified floating-point arithmetic,” arXiv:2511.12294v2, 2025. <https://arxiv.org/abs/2511.12294>

- [4] C. Chang, C. Park, J.-H. Lim, S. Nagarakatte, “P3109 FLoPS: A Lean 4 formalization of IEEE P3109 floating-point semantics,” arXiv:2602.15965, 2026. <https://arxiv.org/abs/2602.15965>
- [5] B. Rouhani et al., “Microscaling data formats for deep learning,” arXiv:2310.10537, 2023. <https://arxiv.org/abs/2310.10537>
- [6] A. Ouyang et al., “KernelBench: Can LLMs write efficient GPU kernels?,” arXiv:2502.10517, 2025. <https://arxiv.org/abs/2502.10517>
- [7] NVIDIA, “Introducing NVFP4 for efficient and accurate low-precision inference,” NVIDIA Technical Blog, 2025. <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>
- [8] M. Wang et al., “ M^2 XFP: A unified mixed-precision microscaling floating-point representation for next-generation AI accelerators,” arXiv:2601.19213, 2026 (to appear, ASPLOS ’26). <https://arxiv.org/abs/2601.19213>
- [9] N. J. Higham, “Low-precision floating-point formats: The wild west of computer arithmetic,” SIAM News, 2017. <https://www.siam.org/publications/siam-news/articles/low-precision-floating-point-formats-the-wild-west-of-computer-arithmetic/>
- [10] E. Carson, X. Chen, “PychoP: Emulating low-precision arithmetic in Python for ML and scientific computing,” arXiv:2504.07835, 2025. <https://arxiv.org/abs/2504.07835>
- [11] C. M. Wintersteiger, “Floating-point conformance testing in industrial practice,” Proc. IEEE ARITH 2025. <https://arith2025.org/proceedings/215900a157.pdf>
- [12] C. Hunhold, “libtakum: A reference C library for takum arithmetic,” GitHub, 2024–2025. <https://github.com/takum-arithmetic/libtakum>
- [13] C. Hunhold, T. Quinlan, “Takum arithmetic in sparse iterative solvers: A precision-vs-storage study,” arXiv:2412.20268, 2024. <https://arxiv.org/abs/2412.20268>
- [14] B. Nouné et al., “8-bit numerical formats for deep neural networks,” arXiv:2206.02915, 2022. <https://arxiv.org/abs/2206.02915>
- [15] IEEE Std 754-2019, *IEEE Standard for Floating-Point Arithmetic*, Institute of Electrical and Electronics Engineers, New York, NY, 2019. <https://standards.ieee.org/ieee/754/6210/>
- [16] Open Compute Project, *OCP Microscaling Formats (MX) Specification v1.0*, Open Compute Project Foundation, 2023. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
- [17] Google/JAX, `ml_dtypes` version 0.5.4: Low-precision float types for NumPy and JAX, 2024. https://github.com/jax-ml/ml_dtypes
- [18] IEEE P3109 Working Group, *IEEE SA P3109 Interim Report v3.2.0: Standard for Floating-Point Arithmetic for AI*, IEEE Standards Association, 2026. <https://github.com/P3109/Public>